Selecting reliable mathematical solutions

DASE7506 Mini-Project 2

31 October 2026

Abstract

This project studies how to select a correct answer from four supplied mathematical solutions. The method combines a relative linear verifier with two compact neural verifiers trained using different outcome objectives. All model parameters and preprocessing statistics are fitted on the supplied training split. The submitted model answers 399 of 467 test questions correctly, giving 85.4390% selection accuracy. Complete CPU prediction and independent scoring take 3.136 seconds on a Xeon Platinum 8368Q, with 11.26 MiB of inference assets. Experiments compare independent and joint candidate models, training objectives and component combinations.

1 Task and experimental setup

Each question has four candidate solutions and a fixed 1,024-dimensional representation for each candidate. The program returns one candidate index. The official scorer checks its final numeric answer, including equivalent fractions and decimals; any correct candidate earns the point.

The question investigated here is whether learned interactions among candidates can complement a relative linear verifier. The linear model already compares candidates through relative features. The hypothesis is that learned interaction and different outcome objectives can provide complementary rankings.

Split	Questions	Disagreement	Use
Training	4,755	2,114	Parameter and preprocessing fitting
Validation	798	355	Early stopping and model selection
Test	467	467	Official score

A disagreement question has at least two distinct valid parsed answers. Training uses the 2,114 disagreement groups; all fitted normalization and whitening statistics come from these groups. The final CPU model scores 308/355 on validation disagreements and 751/798 on full validation. The full split includes 443 unanimous groups, so its 94.11% accuracy is reported separately from disagreement accuracy.

Validation is used for epoch, seed, regularization and combination choices. Earlier fixed-version test totals were available during development. No individual test errors were used to construct features; the final checkpoint and combination coefficients were fixed before its reported test evaluation.

2 Method

2.1 Relative linear verifier

Groupwise ranking studies candidate scores that depend on other items in the same group [1]. Here, the linear baseline represents candidate context through relative features. Let e_i be candidate i's representation. Its relative representation is

$$c_i = e_i - \frac{1}{4}\sum_{j=1}^{4} e_j, \qquad r_i = \frac{c_i}{\frac{1}{4}\sum_{j=1}^{4}\|c_j\|_2 + 10^{-6}}.$$

There are also 32 scalar features. Ten summarize checkable arithmetic, such as the proportion of incorrect equalities and whether the final answer appears in a calculation. The remaining 22 describe literal number sources and answer properties, including numbers appearing in the question or previous calculations, new numbers, sign, magnitude and denominator. Each scalar is centered and scaled within its question. Training-column statistics then standardize the combined 1,056 features.

Candidate labels indicate whether the parsed answer matches the training reference. The linear model minimizes binary cross-entropy with L2 shrinkage, as in ridge regularization [2], using separate penalties of 0.05 for representation coefficients and 0.003 for scalar coefficients. Float64 L-BFGS runs for at most 350 iterations. Six regularization pairs were fitted, and count bonuses of 0, 0.5 and 1 were compared on validation.

Self-consistency uses consensus among final answers to select a solution [3]. For the four fixed candidates here, candidates sharing a parsed answer form a group. For answer a, the selected group score is

$$L(a) = \max_{i:\,\hat{a}_i = a} s_i + 0.5 \log n_a,$$

where s_i is the linear logit and n_a is the number of candidates with answer a. The count term supports repeated answers while allowing a high-scoring minority answer to be selected.

2.2 Neural verifiers

The neural models standardize the representations and apply full-dimensional ZCA whitening with ridge 0.1. They read the transformed vector, its centered and normalized version within the question, and the same scalar features. Preprocessed values are clipped to $[-6, 6]$.

Eight members share projection matrices and have member-specific input and output scales, following the efficient-ensemble idea in TabM [4]. Each member has width 128. The joint model uses one self-attention block over the four candidates, a learned same-answer bias, residual connections, layer normalization and a 128–256–128 feedforward block. Candidate positions have no learned embeddings. This set-based comparison follows the motivation of Set Transformer [5].

For a question, let V contain valid candidates and C contain correct candidates. The two submitted neural objectives are

$$\mathcal{L}_{\text{mass}} = \operatorname{LSE}_{i\in V} s_i - \operatorname{LSE}_{i\in C} s_i,$$
$$\mathcal{L}_{\text{pair}} = \frac{1}{|C|\,|V \setminus C|} \sum_{p\in C} \sum_{n\in V\setminus C} \log(1 + \exp(s_n - s_p)).$$

Both add 0.1 times valid-candidate BCE and average losses over members and questions. The mass objective assigns probability to the correct set; the pairwise objective compares correct and incorrect candidates.

Training uses AdamW with learning rate 0.001, weight decay 0.03, dropout 0.25 and batches of 128 questions. The maximum is 150 epochs, with patience 20. Seeds 17, 29 and 43 were compared. Early stopping uses validation disagreement accuracy, with loss breaking ties. The submitted mass model uses seed 17 and epoch 30; the pairwise model uses seed 29 and epoch 38.

2.3 Final selection

Let M and P denote neural logits averaged over the eight members. Let Z standardize valid candidate scores within the current question, with a standard-deviation floor of 10^{-6}. The combination is

$$B = 0.5Z(L) + 0.5Z(M),$$
$$S = 0.5Z(B) + 0.5Z(P).$$

The second normalization is applied to B before the final combination. The program returns the highest-scoring candidate, excluding invalid answers whenever a valid candidate exists.

3 Results and component comparisons

All development comparisons use the same training and validation splits. Table 1 reports selected settings rather than averages over seeds. Neural development scores use saved GPU outputs; the final row is the deployed CPU implementation.

Method	Correct / 355	Accuracy
Relative linear verifier L	305	85.92%
Independent candidate network	308	86.76%
Joint mass model M	305	85.92%
Joint pairwise model P	305	85.92%
Joint answer-group objective	308	86.76%
Combination $B = L + M$	309	87.04%
Final combination, development scores	309	87.04%
Final combination, CPU deployment	308	86.76%

Table 1: Validation disagreement results.

Combining L and M increases the development count from 305 for either component to 309. Removing either component gives the corresponding standalone result, providing a component ablation. The independent and answer-group networks each reach 308. These are comparisons of complete selected configurations; changes in seed and epoch accompany the architecture and objective choices.

The final combination ties B at 309 using development scores. CPU deployment changes one close validation choice, giving 308/355; the maximum combined-score difference is 6.79×10^{-7}. This was recorded before test evaluation, with weights and coefficients unchanged.

The final candidate was chosen from existing validation-tied combinations, using model size and then name to break ties. Table 2 lists the official complete-test records of the fixed versions.

The submitted combination answers three additional test questions correctly compared with B. Its validation count and CPU/GPU difference are reported alongside that result, so the comparison retains the distinction between development scoring and deployment.

Version	Correct / 467	Accuracy
Earlier Gaussian/path model	385	82.4411%
Two-model combination B	396	84.7966%
Submitted final combination	399	85.4390%

Table 2: Official test accuracy for fixed evaluated versions.

4 Resources and reproducibility

The complete CPU measurement includes startup, imports, loading, preprocessing, every ensemble member and a separate scoring process. CUDA is hidden, numerical libraries use one thread and CPU affinity is restricted to four cores. The local processor is a Xeon Platinum 8368Q; official timing uses the reference Xeon Platinum 8457C.

Measurement	Recorded value	Course limit
Complete prediction and scoring	3.136154 s	30 s
Uncompressed inference assets	11,808,974 bytes	16 MiB
Observed total threads, including monitor	2	4

The predictor uses NumPy. Its ten asset files contain all weights, configurations and learned statistics, including both stored ZCA transforms. Compared with B, assets increase from 5.64 to 11.26 MiB and evaluation time from 2.696 to 3.136 seconds. The final configuration remains within the local resource thresholds.

Recent development experiment	Fits	Stage time (s)
Linear regularization pairs	6	21.72
Independent candidate networks	3	56.33
Joint mass networks	3	55.85
Joint pairwise networks	3	53.27
Joint answer-group networks	3	80.96

Each training run uses one RTX 4090. Two independent experiments ran concurrently, one per card; no run used multiple GPUs. Stage times are recorded separately. The supplied training command also reproduces all 18 fits serially on one card in 274.07 seconds. All ten checkpoint files and all 62 arrays matched the released version exactly in the recorded environment. Earlier search records are retained separately because some historical entries overlap.

The repository provides exact installation, training and evaluation commands, dependency versions, seeds, official score JSON, resource records and asset hashes. Checkpoint evaluation requires no retraining. The interface is `predict(bank, arrays, checkpoint)` and returns Python integer indices in input order. The original data, parser and scorer are unchanged.

5 Discussion

The component comparison shows complementary rankings from the linear and mass models. The final pairwise branch contributes three additional correct test selections at an additional 0.440 seconds of local evaluation time and approximately 5.62 MiB of assets. Its validation count remains tied with B, so the reported gain is specific to this course bank.

The labels concern final answers. Arithmetic features measure local numerical consistency, while assigning quantities to the correct objects and times requires further semantic information. For example, an earlier training-error review included a solution that computed $8 \times 5 = 40$ total pieces of wood, then treated 40 as a per-item quantity. This illustrates the distinction between checking an equality and checking how its result is used. The example comes from training analysis of an earlier version, not from new inspection of the final test errors.

Validation contains 355 disagreement questions and supports several model-selection decisions. The comparisons describe the selected configurations on the provided splits; an independent bank would provide an additional measure of generalization.

The neural models adapt ideas from [4, 5]; all parameters are trained from scratch on course data, with no additional pretrained weights or external datasets.

Acknowledgement. OpenAI Codex assisted with implementation, experiment execution, analysis and report preparation.

References

[1] Q. Ai, X. Wang, S. Bruch, N. Golbandi, M. Bendersky and M. Najork. Learning Groupwise Multivariate Scoring Functions Using Deep Neural Networks. ICTIR, pp. 85–92, 2019. https://arxiv.org/abs/1811.04415.

[2] A. E. Hoerl and R. W. Kennard. Ridge Regression: Biased Estimation for Nonorthogonal Problems. Technometrics, 12(1):55–67, 1970. https://doi.org/10.1080/00401706.1970.10488634.

[3] X. Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR, 2023. https://arxiv.org/abs/2203.11171.

[4] Y. Gorishniy, A. Kotelnikov and A. Babenko. TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling. ICLR, 2025. https://arxiv.org/abs/2410.24210.

[5] J. Lee et al. Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks. ICML, PMLR 97:3744–3753, 2019. https://proceedings.mlr.press/v97/lee19d.html.